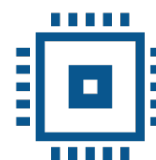




Universitatea *Transilvania* din Brașov
Facultatea de Inginerie Electrică și Știința
Calculatoarelor
Departamentul Automatică și Tehnologia Informației



Temă de semestru P00/PCLP3

Jipa Stefan Ciprian
Gr. 4LF442, AIA II

BRAȘOV, 2026

Cuprins

<u>1. Titlul proiectului.....</u>	<u>2</u>
<u>1.1. Enunț.....</u>	<u>2</u>
<u>1.2. Rezolvare proiect.....</u>	<u>2</u>
<u>1.3. Rezultate.....</u>	<u>3</u>

1. Titlul proiectului

Laborator 10

1.1. Enunț

Exercițiul 1 – Tranzacție de vânzare Se cere implementarea unei operații de vânzare care folosește mecanismul de tranzacții:

- adăugarea tabelii Vanzari cu coloanele Id, ProdusId (FK), Cantitate, PretUnitar, DataOra;
- adăugarea în formular a unui buton „Vinde” care: o citește cantitatea din TextBox; o verifică existența stocului; o decrementează stocul în tabela Produse; o înserează o înregistrare în tabela Vanzari;
- toate cele patru operații (interogare, verificare, update, insert) se vor executa într - o singură SqlTransaction - în caz de eroare se apelează Rollback();
- testarea comportamentului prin simularea unei erori (de exemplu cu o cantitate negativă) și verificarea că stocul rămâne neschimbat.

1.2. Rezolvare proiect

lab10.java

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Windows.Forms;

namespace Laboratoare
{
    public partial class Form1 : Form
    {
        // Șirul de conexiune către LocalDB (implicit în Visual Studio)
        private string masterConnectionString = @"Server=(localdb)
\mssqllocaldb;Database=master;Trusted_Connection=True;";
        private string connectionString = @"Server=(localdb)
\mssqllocaldb;Database=ManagerProduseLab10;Trusted_Connection=True;";

        private DataGridView dgvProduse;
        private TextBox tbCantitate;
        private Button btnVinde;
        private Label lblErori;

        public Form1()
        {
            // Setări formular
            this.Text = "Tranzacție de vânzare - Lab 10";
            this.Size = new Size(600, 450);
            this.StartPosition = FormStartPosition.CenterScreen;

            // Inițializare baza de date
            InitializeazaBazaDeDate();

            // Construire Interfață Grafică
            Label lblTitlu = new Label { Text = "Produse disponibile:", Location =
new Point(20, 20), AutoSize = true, Font = new Font(this.Font, FontStyle.Bold) };
        }
    }
}
```

```

        dgvProduse = new DataGridView
        {
            Location = new Point(20, 45),
            Size = new Size(540, 200),
            SelectionMode = DataGridViewSelectionMode.FullRowSelect,
            MultiSelect = false,
            AllowUserToAddRows = false,
            ReadOnly = true
        };

        Label lblCant = new Label { Text = "Cantitate de vândut:", Location = new
Point(20, 260), AutoSize = true };
        tbCantitate = new TextBox { Location = new Point(130, 257), Width =
100 };

        btnVinde = new Button { Text = "Vinde", Location = new Point(240, 255),
Width = 100, BackColor = Color.LightGreen };
        btnVinde.Click += BtnVinde_Click;

        lblErrori = new Label { Location = new Point(20, 300), Size = new
Size(540, 80), ForeColor = Color.Red };

        this.Controls.AddRange(new Control[] { lblTitlu, dgvProduse, lblCant,
tbCantitate, btnVinde, lblErrori });

        // Încărcare date
        IncarcaProduse();
    }

    private void IncarcaProduse()
    {
        try
        {
            using (SqlConnection con = new SqlConnection(connectionString))
            {
                SqlDataAdapter da = new SqlDataAdapter("SELECT Id, Denumire,
Stoc, PretUnitar FROM Produse", con);
                DataTable dt = new DataTable();
                da.Fill(dt);
                dgvProduse.DataSource = dt;
            }
        }
        catch (Exception ex)
        {
            lblErrori.Text = "Eroare la încărcarea produselor: " + ex.Message;
        }
    }

    private void BtnVinde_Click(object sender, EventArgs e)
    {
        lblErrori.Text = ""; // Curățăm mesajele de eroare anterioare

        if (dgvProduse.SelectedRows.Count == 0)
        {
            MessageBox.Show("Selectați un produs din listă pentru a-l vinde.");
            return;
        }

        if (!int.TryParse(tbCantitate.Text, out int cantitateDovita) ||
cantitateDovita == 0)
        {
            MessageBox.Show("Introduceți o cantitate validă (diferită de 0).
Atenție: puteți introduce o cantitate negativă pentru a testa Rollback-ul la

```

```

eroare!");
        }
        return;
    }

    int produsId =
Convert.ToInt32(dgvProduce.SelectedRows[0].Cells["Id"].Value);
    decimal pretUnitar =
Convert.ToDecimal(dgvProduce.SelectedRows[0].Cells["PretUnitar"].Value);

    using (SqlConnection con = new SqlConnection(connectionString))
    {
        con.Open();
        // 1. Începem tranzacția
        SqlTransaction transaction = con.BeginTransaction();

        try
        {
            // 2. Verificăm stocul curent (Interogare)
            SqlCommand checkCmd = new SqlCommand("SELECT Stoc FROM Produce
WHERE Id = @Id", con, transaction);
            checkCmd.Parameters.AddWithValue("@Id", produsId);

            int stocCurent = (int)checkCmd.ExecuteScalar();

            // 3. Verificarea condiției logice
            if (cantitateDovita < 0)
            {
                throw new InvalidOperationException("Eroare simulată:
Cantitatea nu poate fi negativă! Se va face Rollback.");
            }

            if (stocCurent < cantitateDovita)
            {
                throw new Exception($"Stoc insuficient! Stoc curent:
{stocCurent}, dorit: {cantitateDovita}");
            }

            // 4. Decrementăm stocul (Update)
            SqlCommand updateCmd = new SqlCommand("UPDATE Produce SET Stoc =
Stoc - @Cantitate WHERE Id = @Id", con, transaction);
            updateCmd.Parameters.AddWithValue("@Id", produsId);
            updateCmd.Parameters.AddWithValue("@Cantitate", cantitateDovita);
            updateCmd.ExecuteNonQuery();

            // 5. Inserăm înregistrarea în tabela Vanzari (Insert)
            SqlCommand insertCmd = new SqlCommand(
                "INSERT INTO Vanzari (ProdusId, Cantitate, PretUnitar,
DataOra) VALUES (@ProdusId, @Cantitate, @PretUnitar, @DataOra)",
                con, transaction);
            insertCmd.Parameters.AddWithValue("@ProdusId", produsId);
            insertCmd.Parameters.AddWithValue("@Cantitate", cantitateDovita);
            insertCmd.Parameters.AddWithValue("@PretUnitar", pretUnitar);
            insertCmd.Parameters.AddWithValue("@DataOra", DateTime.Now);
            insertCmd.ExecuteNonQuery();

            // 6. Confirmăm modificările dacă totul a decurs bine
            transaction.Commit();

            MessageBox.Show("Vânzarea a fost înregistrată cu succes!",
"Succes", MessageBoxButtons.OK, MessageBoxIcon.Information);
            tbCantitate.Clear();
            IncarcaProduce(); // Reîncărcăm grila pentru a vedea noul stoc
        }
        catch (Exception ex)
    }

```

```

        {
            // 7. Anulăm toate modificările în caz de eroare (Rollback)
            transaction.Rollback();
            lblErori.Text = "Tranzacție anulată (Rollback) din cauza erorii:
\n" + ex.Message;
        }
    }

    // =====
    // METODĂ AJUTĂTOARE PENTRU CREAREA AUTOMATĂ A BAZEI DE DATE
    // =====
    private void InitializeazaBazaDeDate()
    {
        try
        {
            using (SqlConnection masterCon = new
SqlConnection(masterConnectionString))
            {
                masterCon.Open();

                // Verificăm dacă baza de date există
                SqlCommand checkDb = new SqlCommand("SELECT
db_id('ManagerProduseLab10')", masterCon);
                if (checkDb.ExecuteScalar() == DBNull.Value)
                {
                    // Creăm baza de date
                    SqlCommand createDb = new SqlCommand("CREATE DATABASE
ManagerProduseLab10", masterCon);
                    createDb.ExecuteNonQuery();
                }
            }

            using (SqlConnection con = new SqlConnection(connectionString))
            {
                con.Open();

                // Creăm tabelele dacă nu există
                string createTablesSql = @"
                IF NOT EXISTS (SELECT * FROM sysobjects WHERE name='Produse')
                BEGIN
                    CREATE TABLE Produse (
                        Id INT IDENTITY(1,1) PRIMARY KEY,
                        Denumire NVARCHAR(100) NOT NULL,
                        Stoc INT NOT NULL,
                        PretUnitar DECIMAL(18,2) NOT NULL
                    );

                    INSERT INTO Produse (Denumire, Stoc, PretUnitar) VALUES
                    ('Laptop Dell', 10, 3500.00),
                    ('Mouse Wireless', 50, 120.50),
                    ('Tastatură Mecanică', 30, 250.00);
                END

                IF NOT EXISTS (SELECT * FROM sysobjects WHERE name='Vanzari')
                BEGIN
                    CREATE TABLE Vanzari (
                        Id INT IDENTITY(1,1) PRIMARY KEY,
                        ProdusId INT FOREIGN KEY REFERENCES Produse(Id),
                        Cantitate INT NOT NULL,
                        PretUnitar DECIMAL(18,2) NOT NULL,
                        DataOra DATETIME NOT NULL
                    );
                END";
            }
        }
    }

```

```
        SqlCommand createCmd = new SqlCommand(createTablesSql, con);
        createCmd.ExecuteNonQuery();
    }
}
catch (Exception ex)
{
    MessageBox.Show("Eroare la crearea bazei de date: " + ex.Message);
}
}
}
```

1.3. Rezultate

Tranzacție de vânzare - Lab 10

Produse disponibile:

	Id	Denumire	Stoc	PretUnitar
▶	1	Laptop Dell	3	3500.00
	2	Mouse Wireless	50	120.50
	3	Tastatura Meca...	30	250.00

Cantitate de vândut: